

Relatório Completo: Docker Stellar Testnet + Horizon

Data: 15/07/2026 | **Container:** stellar/quickstart:testing (testnet) | **Ledger:** ~3,629,501

Sumário

1. Especificações da Máquina e Docker
 2. Arquitetura Geral do Sistema
 3. Processo de Inicialização (Startup)
 4. Serviços do Container
 5. Conexões de Rede Externas
 6. Bancos de Dados
 7. Bucket List — Armazenamento de Estado
 8. Fluxo Completo de Sincronização
 9. Diagramas
 10. Conclusão
-

1. Especificações da Máquina e Docker

Máquina Host

Parâmetro	Valor
Sistema Operacional	Microsoft Windows 11 Pro
Processador	AMD Ryzen 7 5700X3D 8-Core (16 logical processors)
Memória RAM Total	31.9 GB (33,437,944 KB)
Memória RAM Livre	11.6 GB (12,154,248 KB)
Docker Desktop	v29.1.3
Docker Compose	v5.0.0-desktop.1
WSL2 Kernel	6.18.33.2-microsoft-standard-WSL2
Docker Engine	16 CPUs, 15.56 GiB total memory

Docker Compose

```
services:
  stellar:
    image: stellar/quickstart:testing
    container_name: stellar-testnet
    ports:
      - "8000:8000"      # Horizon API
      - "11625:11625"    # Stellar Core P2P (peer)
      - "11626:11626"    # Stellar Core HTTP (info/upgrades)
    environment:
      NETWORK: testnet
      ENABLE: core,horizon
      LOG_LEVEL: info
    volumes:
      - stellar-data:/opt/stellar
    stdin_open: true
    tty: true
    restart: unless-stopped

volumes:
  stellar-data:
```

Container

Parâmetro	Valor
Imagem	<code>stellar/quickstart:testing</code> (sha256:ee95e7...)
Container ID	<code>4fbe5ce9b7cd</code>
IP Interno (Docker)	172.18.0.2
Gateway	172.18.0.1
Entrypoint	<code>/start</code>
Driver de Armazenamento	overlayfs
Volumes Montados	<code>stellar-data:/opt/stellar</code> (~5.4 GB de dados)
Restart Policy	unless-stopped

2. Arquitetura Geral do Sistema

A imagem `stellar/quickstart:testing` é uma "bateria inclusa" que empacota **todos** os componentes da Stellar em uma única imagem Docker, gerenciados pelo **Supervisor** (supervisord). Diferentemente de uma instalação production-grade com containers separados, o quickstart roda tudo no mesmo container para simplificar desenvolvimento e testes.

Componentes Instalados

Componente	Versão	Finalidade
Stellar Core (node)	v27.1.0	Participa do consenso na testnet
Stellar Core (captive)	v27.1.0	Apenas para servir dados ao Horizon (não participa do consenso)
Horizon	devel (go1.26.4)	API REST para consulta de dados da rede
PostgreSQL	14 (Alpine)	Banco de dados principal do Horizon
Nginx	—	Reverse proxy para o Horizon (porta 8000)
Supervisor	—	Gerenciador de processos (PID 1)

Árvore de Diretórios do Container

```
/opt/stellar/
├── core/                                # Stellar Core Node
│   ├── bin/start                        # Script de inicialização
│   ├── etc/
│   │   ├── stellar-core.cfg            # Configuração principal
│   │   └── env                        # Variáveis de ambiente do core
│   ├── buckets/                        # Bucket list (estado completo da ledger)
│   ├── stellar.db                      # SQLite (meta-dados do core)
│   ├── stellar-misc.db                 # SQLite miscelânea
│   └── .quickstart-initialized
├── horizon/                            # Horizon API Server
│   ├── bin/start                        # Script de inicialização
│   ├── bin/horizon                    # Binário do Horizon
│   ├── etc/
│   │   ├── horizon.env                # Configuração do Horizon
│   │   └── stellar-captive-core.cfg    # Config do captive core
│   └── captive-core/
│       ├── stellar.db                 # SQLite do captive core
│       ├── stellar-misc.db
│       └── captive-core/
│           ├── stellar-core.conf       # Config gerada do captive core
│           └── buckets/                # Buckets do captive core
├── postgresql/                         # PostgreSQL
│   ├── data/                          # Dados do banco
│   ├── etc/
│   │   ├── postgresql.conf           # Config do PostgreSQL
│   │   ├── pg_hba.conf               # Autenticação
│   │   └── pg_ident.conf
│   └── .pgpass                        # Senha armazenada
├── nginx/                              # Nginx reverse proxy
│   ├── bin/start
│   └── etc/
│       └── nginx.conf
├── supervisor/                         # Supervisor
│   └── etc/
│       └── supervisord.conf
└── stellar-rpc/                        # RPC Server (não ativado no nosso setup)
```

3. Processo de Inicialização

O entrypoint `/start` (script bash de ~600 linhas) orquestra toda a inicialização. O fluxo é:

Fase 1: Configuração de Rede (process_args)

Com base na variável `NETWORK=testnet`, o script define:

```
NETWORK_PASSPHRASE="Test SDF Network ; September 2015"
HISTORY_ARCHIVE_URLS="https://history.stellar.org/prd/core-testnet/core_testnet_001"
```

Fase 2: Copy Defaults (copy_defaults)

Copia arquivos de configuração padrão de `/opt/stellar-default/{common,testnet}/` para os diretórios de cada serviço. Isso só acontece na **primeira execução** (diretórios vazios). Em execuções posteriores, os diretórios já existem (persistidos no volume) e o copy é pulado.

Fase 3: Inicialização do Banco (init_db)

- Gera senha aleatória para PostgreSQL (exibida no log)
- Executa `initdb` no PostgreSQL
- Cria databases: `horizon` (e `core` se `CORE_USE_POSTGRES=true`)
- Cria usuário `stellar` com privilégios
- Para o PostgreSQL (será reiniciado pelo Supervisor depois)

Fase 4: Inicialização do Stellar Core (init_stellar_core)

- Substitui placeholders no `stellar-core.cfg`
- Executa `stellar-core new-db` para criar o schema SQLite
- Cria o arquivo `.quickstart-initialized`

Fase 5: Inicialização do Horizon (init_horizon)

- Substitui placeholders no `horizon.env` e `stellar-captive-core.cfg`
- Executa `horizon db init` para criar as 33 tabelas no PostgreSQL

Fase 6: Inicialização do Supervisor (exec_supervisor)

Prioridade	Serviço	Autostart	Descrição
10	postgresql	false	Banco de dados
20	stellar-core	false	Nó de consenso
30	horizon	false	API server
50	nginx	true	Reverse proxy

Fase 7: Upgrade Local e Monitoramento

- `upgrade_local()` : Apenas para rede `local`
- `service_status()` : Loops que monitoram o status de cada stellar-core e horizon
- `start_optional_services()` : Inicia postgresql → stellar-core → horizon (nesta ordem)

4. Serviços do Container

4.1 Supervisor

PID	1
Comando	<code>/usr/bin/python3 /bin/supervisord -n -c /opt/stellar/supervisor/etc/supervisord.conf</code>
Porta	9001 (localhost, interface HTTP)
Função	Process manager. Mantém todos os serviços rodando, reinicia se necessário.

4.2 Stellar Core — Nó de Consenso (Node)

PID	291
Usuário	stellar
Comando	<code>/usr/bin/stellar-core --conf /opt/stellar/core/etc/stellar-core.cfg run</code>
Uso de Memória	~2.8 GB (17.6% do total do container)
CPU	~12% (pode chegar a 100% durante catchup)
Porta P2P	11625 (escuta 0.0.0.0) — comunicação com peers
Porta HTTP	11626 (escuta 0.0.0.0) — API de info/upgrades

Parâmetros da Configuração

Parâmetro	Valor	Significado
<code>CATCHUP_RECENT</code>	100	Quantos ledgers recentes pegar via history archive
<code>UNSAFE_QUORUM</code>	true	Aceita quorum mesmo sem configuração de nós confiáveis (safe para testnet)
<code>FAILURE_SAFETY</code>	1	Tolerância a falhas (f = 1, precisa de 3 validadores)
<code>MANUAL_CLOSE</code>	false	Fechamento automático de ledger (não manual)
<code>DATABASE</code>	sqlite3://...	Banco de dados local, não usa PostgreSQL

4.3 Stellar Core — Captive Core (Horizon)

PID	3985
Comando	<code>/usr/bin/stellar-core --conf .../stellar-core.conf --console run --metadata-output-stream fd:3</code>
Uso de Memória	~2.9 GB (17.9%)
Porta P2P	11725 (escuta 0.0.0.0)
Porta HTTP	11726 (escuta localhost)

Diferenças: Node vs Captive Core

Característica	Node (PID 291)	Captive Core (PID 3985)
Participa do consenso?	Sim	Não
Conecta a peers?	Sim (P2P)	Sim (apenas catchup inicial)
Database	SQLite (<code>stellar.db</code>)	SQLite (<code>captive-core/stellar.db</code>)
Bucket list	Compartilhada	Isolada (própria)
Porta HTTP	11626 (pública)	11726 (localhost)
Iniciado por?	Supervisor	Horizon (subprocesso)
Ciclo de vida	Permanente	Efêmero
Modo de execução	<code>run</code> (normal)	<code>--console run --metadata-output-stream fd:3</code>

4.4 Horizon API Server

PID	349
Uso de Memória	~182 MB (1.1%)
Porta	8001 (interna), 8000 via Nginx

Configuração (horizon.env)

```
export DATABASE_URL="postgres://stellar:s9YVDakFZwTcrb7h@localhost/horizon"
export STELLAR_CORE_URL="http://localhost:11726"
export STELLAR_CORE_BINARY_PATH=/usr/bin/stellar-core
export LOG_LEVEL="info"
export ENABLE_CAPTIVE_CORE_INGESTION="true"
export CAPTIVE_CORE_USE_DB=true
export INGEST="true"
export PER_HOUR_RATE_LIMIT="72000"
export NETWORK_PASSPHRASE="Test SDF Network ; September 2015"
export HISTORY_ARCHIVE_URLS="https://history.stellar.org/prd/core-testnet/core_testnet_001"
export ADMIN_PORT=6060
export PORT=8001
export CHECKPOINT_FREQUENCY=64
export INGEST_DISABLE_STATE_VERIFICATION=True
export CAPTIVE_CORE_CONFIG_PATH=/opt/stellar/horizon/etc/stellar-captive-core.cfg
export CAPTIVE_CORE_STORAGE_PATH=/opt/stellar/horizon/captive-core
export STELLAR_CORE_VERSION="v27.1.0"
```

4.5 PostgreSQL

Versão	14 (Alpine)
PID	330
Uso de Memória	~25 MB (processo principal)
Porta	5432
max_connections	150
shared_buffers	128 MB

4.6 Nginx

PID	288 (master), 289 (worker)
Porta	8000 (externa)
Função	Reverse proxy. Encaminha requisições da porta 8000 para o Horizon (porta 8001).

5. Conexões de Rede Externas

Conexões Ativas (ESTABLISHED)

IP do Peer	Porta	Status
13.223.55.158	11625	ESTAB
44.204.146.210	11625	ESTAB
3.85.105.105	11625	ESTAB
44.206.255.84	11625	ESTAB
44.213.67.110	11625	ESTAB
98.91.174.101	11625	ESTAB (2 conexões)
141.98.219.89	11625	ESTAB
54.159.155.163	11625	ESTAB
18.220.162.149	11725	ESTAB (captive core)

Validadores da Testnet (SDF)

Nome	Chave Pública	Endereço
sdf_testnet_1	GDKXE20ZMJI...	core-testnet1.stellar.org
sdf_testnet_2	GCUCJTIYXS0X...	core-testnet2.stellar.org
sdf_testnet_3	GC2V2EFSXN6...	core-testnet3.stellar.org

Endpoints de History Archive

URL	Descrição
<code>http://history.stellar.org/prd/core-testnet/core_testnet_001/{0}</code>	Archive 1 (SDF)
<code>http://history.stellar.org/prd/core-testnet/core_testnet_002/{0}</code>	Archive 2 (SDF)
<code>http://history.stellar.org/prd/core-testnet/core_testnet_003/{0}</code>	Archive 3 (SDF)

6. Bancos de Dados

6.1 PostgreSQL (Horizon)

Database: horizon — **Usuário:** stellar — **Tamanho total:** ~1.2 GB

33 Tabelas (com tamanhos)

Tabela	Tamanho	Função
accounts	497 MB	Contas com saldos e sequências
accounts_signers	428 MB	Signers de cada conta
accounts_data	94 MB	Data entries das contas
trust_lines	107 MB	Trust lines (saldo de assets não-nativos)
offers	12 MB	Ofertas em aberto
exp_asset_stats	23 MB	Estatísticas de assets
history_transactions	10 MB	Transações históricas
history_operations	3.3 MB	Operações históricas
claimable_balances	5 MB	Claimable balances
contract_asset_balances	6 MB	Balances de contratos Soroban
history_ledgers	320 kB	Metadados de ledgers
history_effects	536 kB	Efeitos de operações
liquidity_pools	872 kB	Pools de liquidez
... e mais 20 tabelas auxiliares e de junção		

6.2 SQLite (Stellar Core Node)

stellar.db	~18 MB (com WAL: ~62 MB)
stellar-misc.db	~22 MB (com WAL)

6.3 SQLite (Captive Core - Horizon)

captive-core/stellar.db	~21 MB (com WAL: ~33 MB)
captive-core/stellar-misc.db	~14 MB (com WAL)

7. Bucket List — Armazenamento de Estado

A **Bucket List** é o mecanismo de armazenamento de estado do Stellar Core. Diferente de um banco de dados tradicional, ela é uma estrutura de dados **imutável e baseada em merge** que armazena o estado completo da ledger (contas, saldos, offers, trust lines, etc.) em arquivos XDR chamados "buckets".

Características

- **Imutável:** Buckets nunca são modificados após criados. Mudanças geram NOVOS buckets.
- **Níveis (Levels):** A bucket list tem 11 níveis (0-10). Cada nível contém buckets que representam snapshots do estado em diferentes escalas de tempo.
- **Merge:** Periodicamente, buckets de níveis inferiores são mergeados em níveis superiores (compaction).
- **Hashes:** Cada bucket é identificado por um hash SHA-256 do seu conteúdo.
- **Formato:** Arquivos `.xdr` (XDR serialization) com arquivos `.index` para lookup.

Tamanho Total

Node Core	~4.7 GB	<code>/opt/stellar/core/buckets/</code>
Captive Core	~5.4 GB	<code>/opt/stellar/horizon/captive-core/captive-core/buckets/</code>

Maiores Buckets

Bucket	Tamanho
<code>bucket-ff26b4f5...xdr</code>	822 MB
<code>bucket-eb7625ce...xdr</code>	639 MB
<code>bucket-d8522ede...xdr</code>	629 MB
<code>bucket-8a27deda...xdr</code>	473 MB
<code>bucket-b2a2c09a...xdr</code>	371 MB

8. Fluxo Completo de Sincronização

Timeline Real (do nosso container)

Tempo	Evento	Ledger
00:57:23	Container inicia	—
00:57:30	PostgreSQL disponível	—
00:57:33	stellar-core node inicia	—
00:57:35	Horizon inicia (modo ingestão)	—
00:57:35	Catching up — ETA: 280s	3,629,377
00:57:35	Download de state files + buckets	3,629,311
00:59:xx	Download de buckets: 8% → 70%	—
01:04:xx	Download completo → Applying buckets	—
01:05:xx	Succeeded: download-verify-apply-buckets	3,629,309
01:05:xx	Download de checkpoints restantes	—
01:05:xx	Captive Core Connected	3,629,376
01:08:xx	Node Synced!	3,629,436
01:09:xx	Horizon core: downloading ledger files	3,629,375
01:11:23	Horizon: ingestion caught up	3,629,501
01:11:48	Core latest ledger	3,629,501

Tempo total para sincronizar: ~14 minutos (primeira execução, com buckets quentes em cache)

9. Diagramas

Diagrama 1: Bancos de Dados (ER)

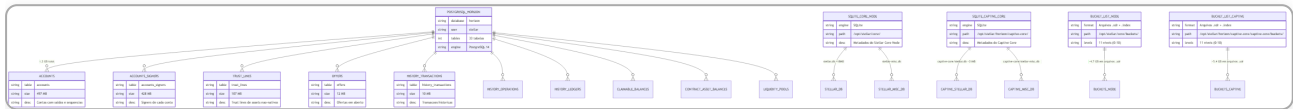
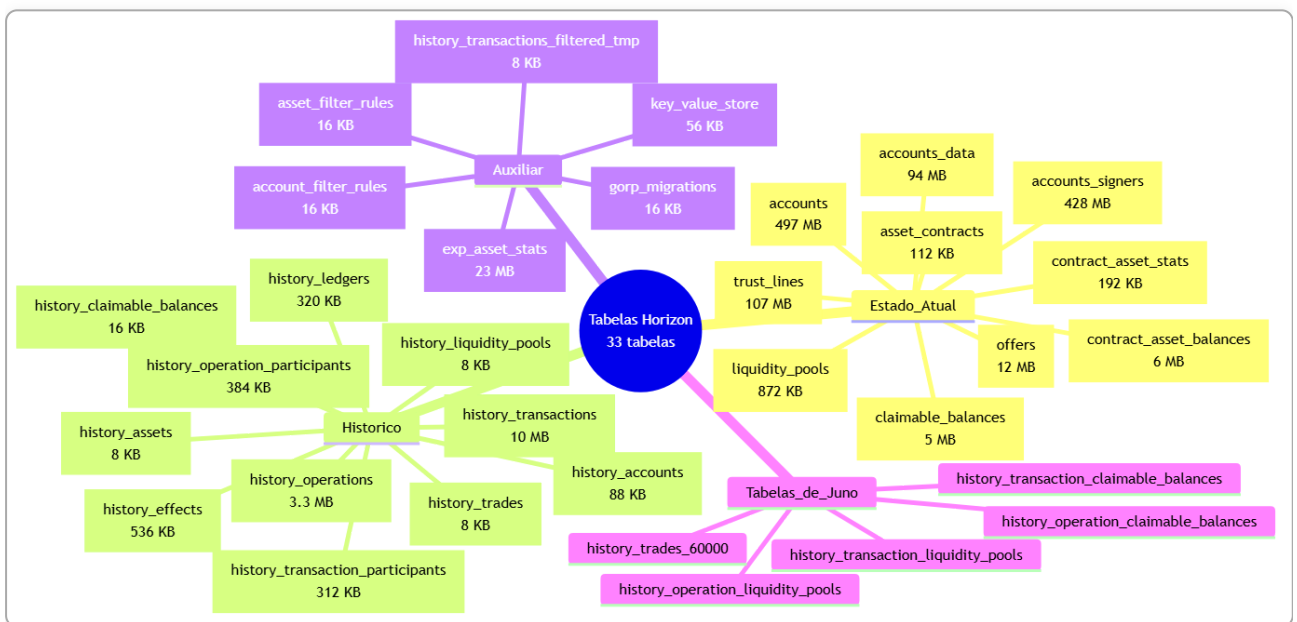


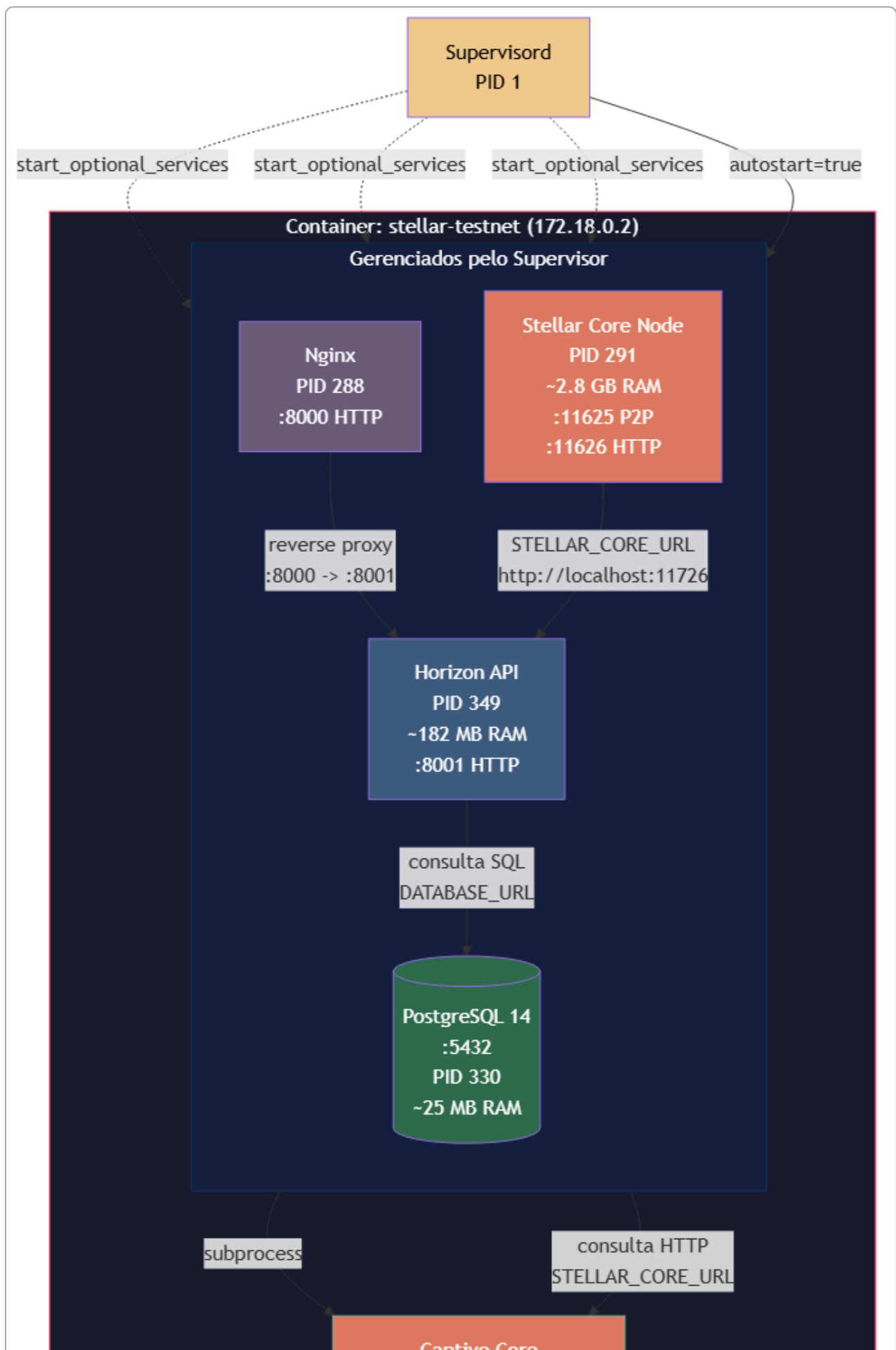
Diagrama entidade-relacionamento dos bancos: PostgreSQL (Horizon), SQLite (Node e Captive Core) e Bucket Lists.

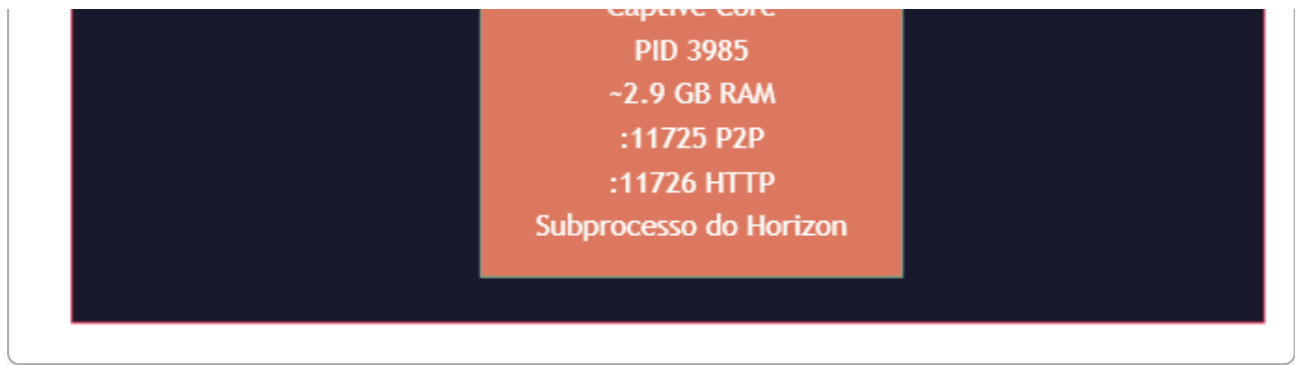
Diagrama 2: Tabelas do Horizon (Mindmap)



Mindmap das 33 tabelas do PostgreSQL organizadas por categoria: Estado Atual, Histórico, Auxiliares e Junção.

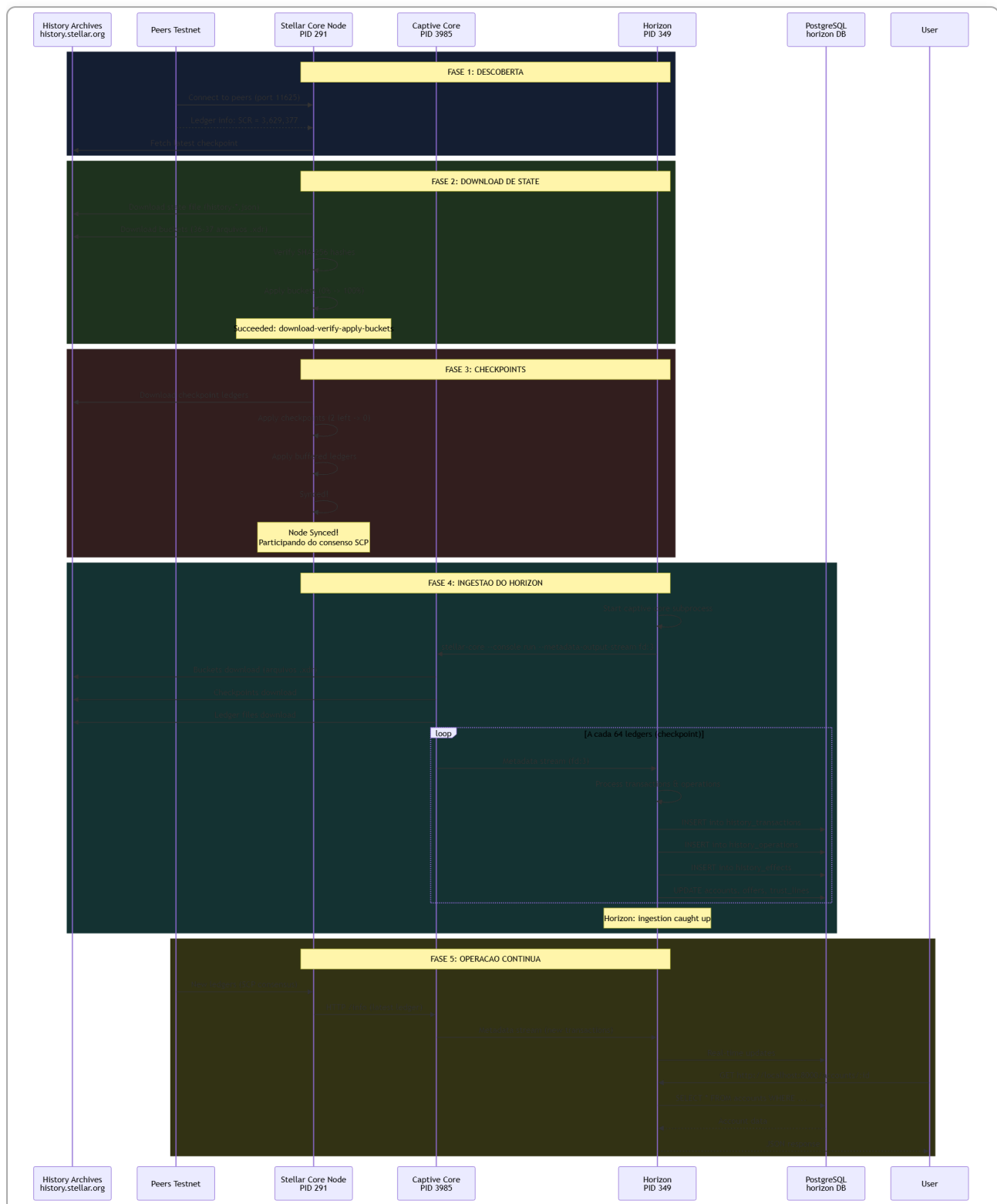
Diagrama 3: Serviços do Container





Arquitetura de serviços dentro do container: Supervisor gerenciando PostgreSQL, Stellar Core Node, Horizon e Nginx, com Captive Core como subprocesso do Horizon.

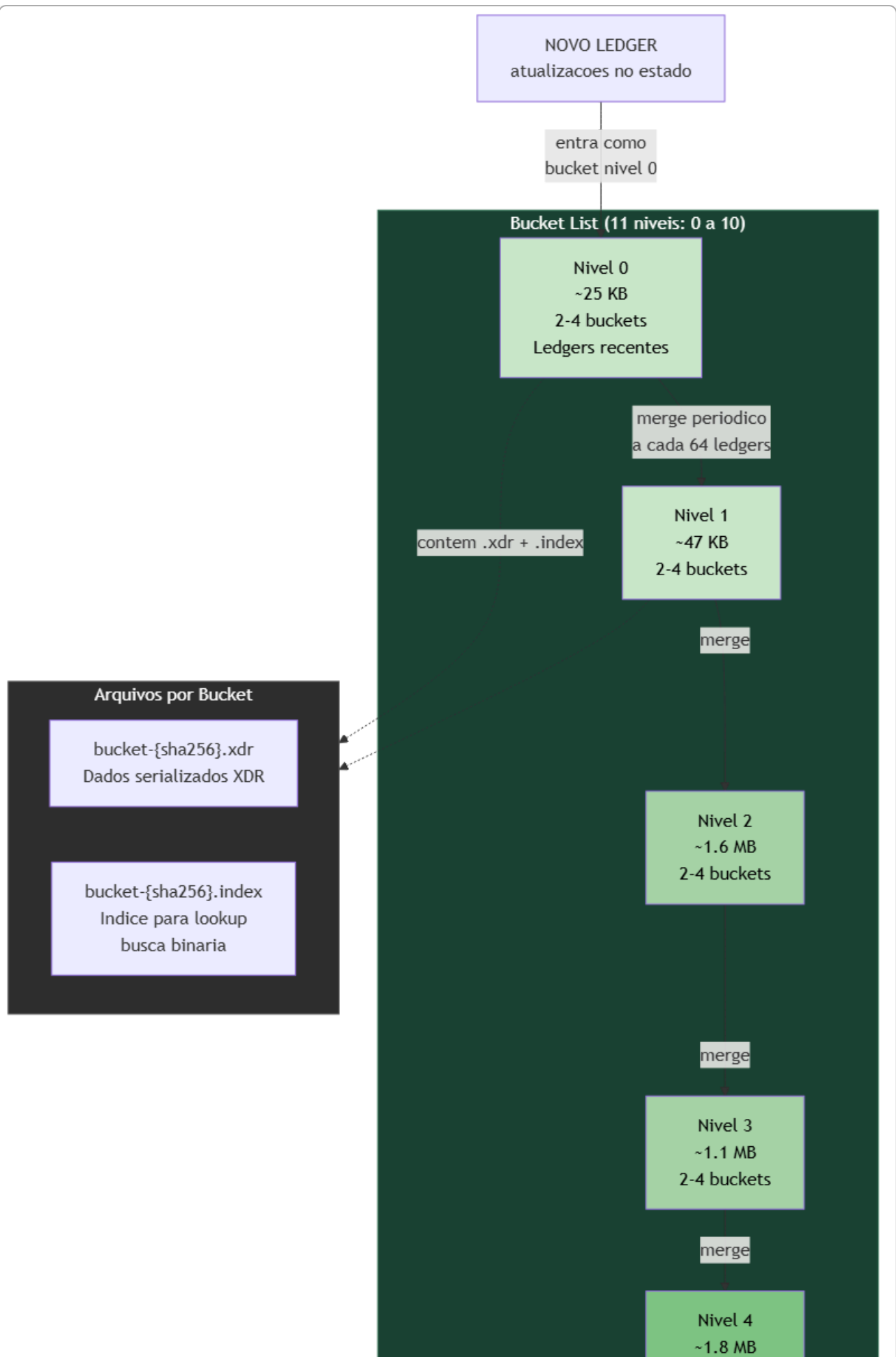
Diagrama 4: Fluxo de Sincronização (Sequência)

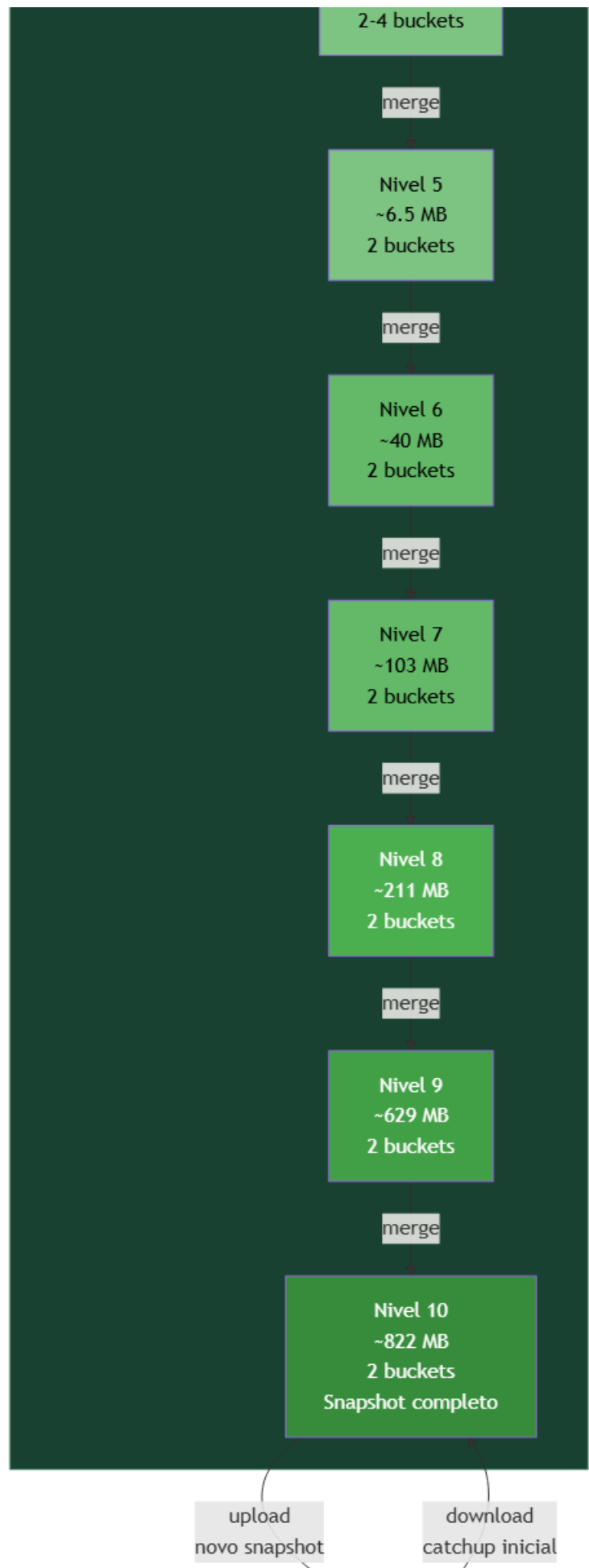


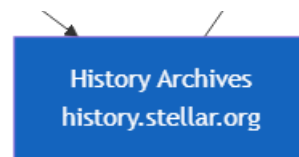
Fluxo completo de sincronização em 5 fases: Descoberta, Download de State, Checkpoints, Ingestão do Horizon e Operação Contínua.

Diagrama 5: Hierarquia da Bucket List



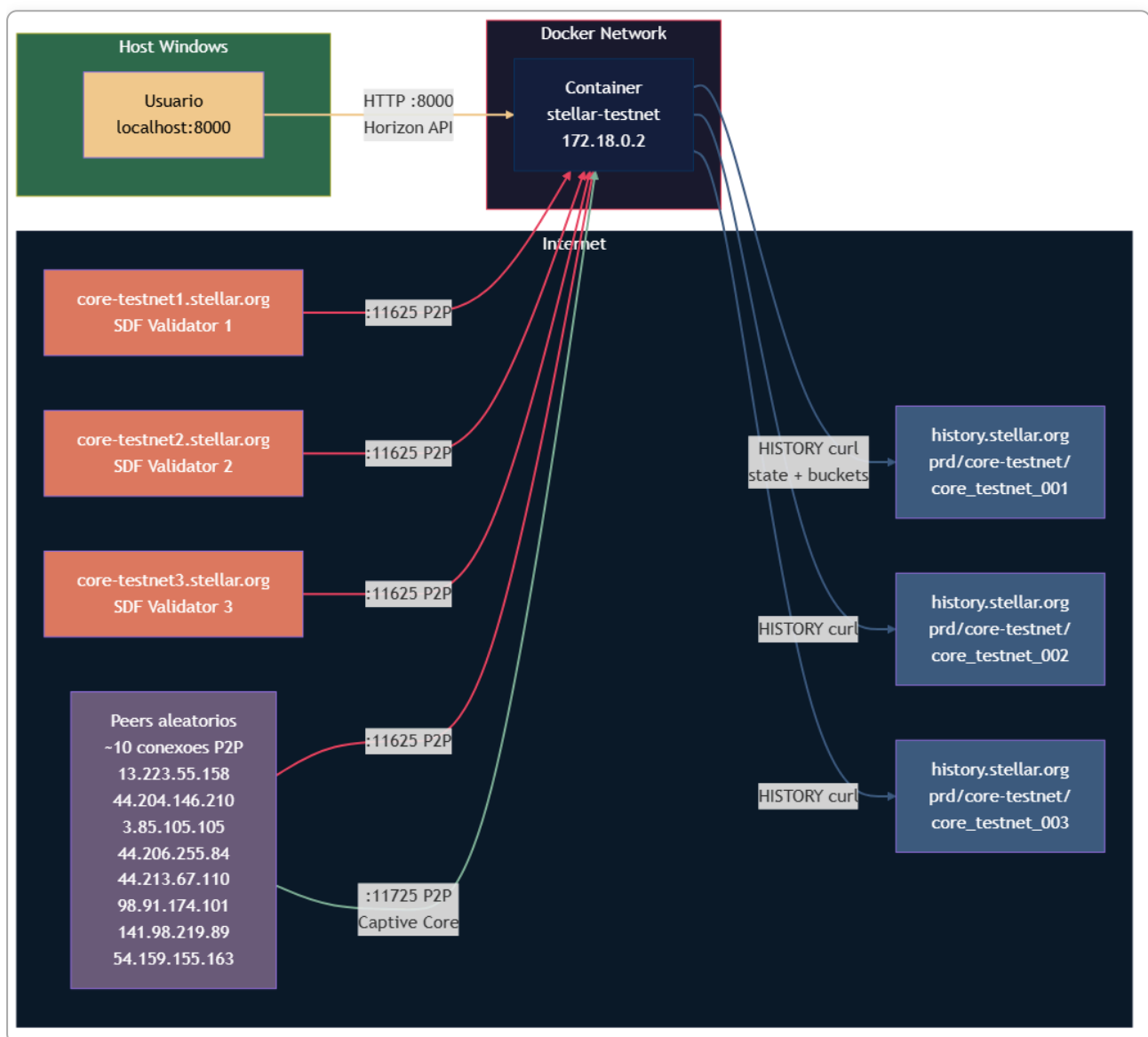






Hierarquia dos 11 níveis da Bucket List (0-10) com tamanhos, processo de merge e conexão com History Archives.

Diagrama 6: Conexões de Rede Externas



Conexões de rede do container com validadores SDF, peers P2P, History Archives e o usuário local.

10. Conclusão

Resumo dos Dados de Armazenamento

Componente	Tecnologia	Tamanho	Localização
Bucket List (Node)	Arquivos .xdr	4.7 GB	<code>/opt/stellar/core/buckets/</code>
Bucket List (Captive)	Arquivos .xdr	5.4 GB	<code>/opt/stellar/horizon/captive-core/captive-core/buckets/</code>
SQLite (Node)	SQLite	62 MB (com WAL)	<code>/opt/stellar/core/stellar.db</code>
SQLite (Captive)	SQLite	33 MB (com WAL)	<code>/opt/stellar/horizon/captive-core/stellar.db</code>
PostgreSQL (Horizon)	PostgreSQL	~1.2 GB	Volume Docker (<code>/opt/stellar/postgresql/data/</code>)
Total		~10-11 GB	Volume Docker <code>stellar-data</code>

Por que 2 instâncias do Stellar Core?

O uso de **duas instâncias** do stellar-core (node + captive) é uma arquitetura deliberada do Horizon moderno:

1. **Node (PID 291):** Mantém o estado completo da rede, participa do consenso SCP, conecta-se a peers. É o "verificador" que garante que os dados estão corretos.
2. **Captive Core (PID 3985):** É uma instância **descartável e isolada** que o Horizon gerencia. Ela não participa do consenso — apenas faz catchup via history archives e envia metadados das transações para o Horizon via pipe (`fd:3`).

Como o Horizon Sabe o que Está Acontecendo?

O Horizon se conecta ao Captive Core via:

1. **HTTP** (`STELLAR_CORE_URL=http://localhost:11726`) — informações de ledger atual

2. **Metadata Stream** (`--metadata-output-stream fd:3`) — pipe para metadados de transações em tempo real
3. **SQLite Database** (`CAPTIVE_CORE_USE_DB=true`) — acesso direto ao banco do captive core

Comandos Úteis

```
# Ver logs do stellar-core node
docker compose logs stellar-testnet | grep "stellar-core(node)"

# Ver logs do captive core
docker compose logs stellar-testnet | grep "stellar-core(horizon)"

# Ver logs do horizon
docker compose logs stellar-testnet | grep "horizon:"

# Ver status atual via API
curl -s http://localhost:8000 | jq '.core_latest_ledger, .history_latest_ledger'

# Listar peers conectados
curl -s http://localhost:11626/peers | jq '.outgoing_peers[] | {ip, port, lat}'

# Ver info do stellar-core node
curl -s http://localhost:11626/info | jq '.info.state, .info.ledger.num, .info.peers'
```

Relatório gerado em: 15/07/2026

Baseado no container: stellar/quickstart:testing (testnet)

Ledger no momento da coleta: ~3,629,501